



Universidade de Brasília
Instituto de Exatas
Departamento de Ciência da Computação

Relatório do Laboratório 3

Processador Multiciclo

CIC0099 - Organização e Arquitetura de Computadores

Turma 02 - Prof. Dr. Ricardo Pezzuol Jacobi

Grupo B4

Henrique Morcelles Salum	232003008
Athos Calixto Muniz	211068261
Gabriela Fernanda Rodrigues Costa	180120859
Cauê Araújo Euzebio	211028195
Lucas Silva Nóbrega	180035096

Sumário

1	Introdução	2
2	Métodos e Análise	2
2.1	Experimento	2
2.1.1	Tarefa I	3
2.1.2	Tarefa II	3
2.1.3	Tarefa III	3
2.2	Tarefa IV	6
2.2.1	Tarefa V	6
2.2.2	Tarefa VI	6
2.3	Tarefa VII	6

1 Introdução

O presente experimento tem como objetivo introduzir os alunos ao processo de projeto e implementação de sistemas digitais utilizando VHDL como Linguagem de Descrição de Hardware (HDL). A atividade busca promover o treinamento prático em modelagem, análise e síntese de circuitos digitais, utilizando o ambiente de desenvolvimento **Intel Quartus Prime v24.1** como ferramenta principal.

Como aplicação prática, propõe-se a implementação de um processador Multiciclo compatível com a ISA RV32I reduzida, contendo as instruções: `add`, `sub`, `and`, `or`, `slt`, `lw`, `sw`, `beq`, `jal`, `jalr`, `addi` e `lui`.

A implementação será validada por meio de simulação funcional e temporal, com análise de *timing* e frequência máxima de operação, permitindo compreender, de forma integrada, o funcionamento de uma arquitetura RISC-V.

2 Métodos e Análise

2.1 Experimento

Enunciado

Implemente o processador Multiciclo com ISA Reduzida com as instruções: **add**, **sub**, **and**, **or**, **slt**, **lw**, **sw**, **beq**, **jal**, e ainda as instruções **jalr**, **addi** e **lui**.

Para realizar o solicitado, é-nos fornecido o esquemático de uma implementação do Multiciclo com ISA reduzida. Nesse esquemático, porém, foi necessário realizar alterações no caminho de dados, que serão devidamente justificadas adiante.

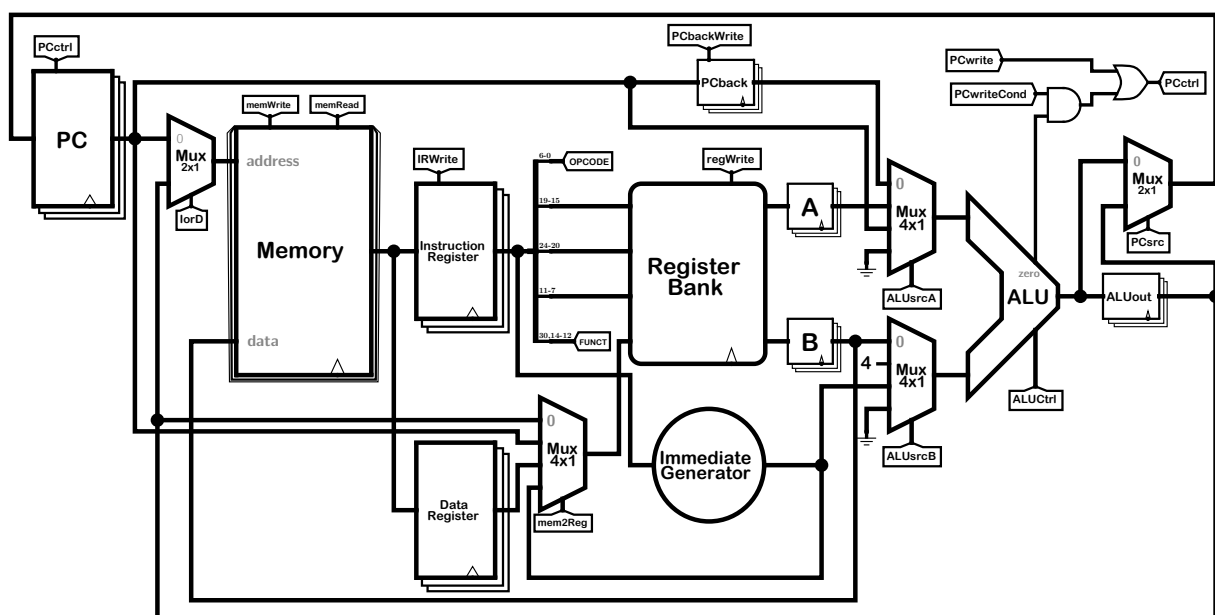


Figura 1: Esquemático do Multiciclo com ISA reduzida

Em que os túneis representam entradas e saídas do módulo de controle do processador, que foi abstraído na figura acima e será devidamente explicado na seção adequada.

2.1.1 Tarefa I

Enunciado

2.1.2 Tarefa II

Enunciado

2.1.3 Tarefa III

Enunciado

Implemente o Bloco Controlador e desenhe a máquina de estados do controle.

Como o enunciado adianta, o bloco de controle do multiciclo é implementado via máquina de estado. Particularmente, utilizamos uma máquina de estados finita do tipo Moore (isto é, em que a saída depende exclusivamente do estado atual, não das entradas). Para projetar uma máquina de estados, primeiro precisamos do seu diagrama de estados; o diagrama de estados completo do bloco de controle do multiciclo com ISA reduzida, considerando as doze instruções solicitadas, é mostrado na Figura 2.

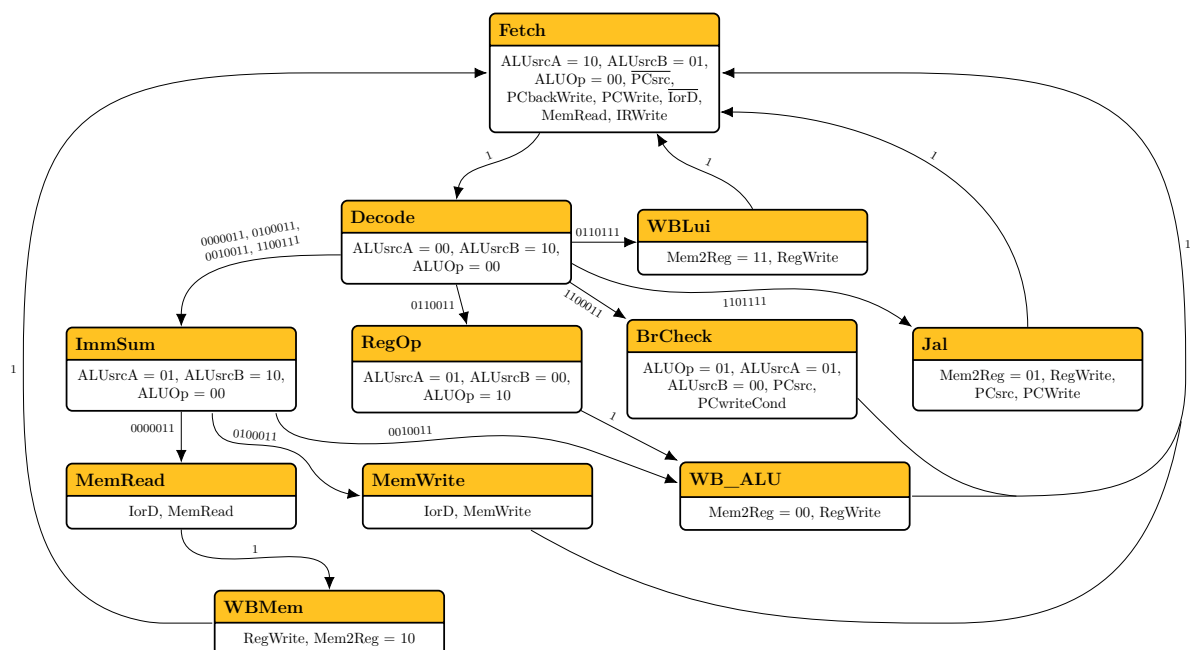


Figura 2: Diagrama de estados do controle

A partir desse diagrama, conseguimos montar as tabelas (1) de saída e (2) de transição de estados, a partir das quais é possível inferir as lógicas de saída e de próximo estado, respectivamente.

Estado Atual	Saídas													
	PCsrc	ALUOp	ALUSrcA	ALUSrcB	PCbackWrite	RegWrite	Mem2Reg	PCwriteCond	PCWrite	IorD	MemWrite	MemRead	IRWrite	
0000	0	00	10	01	1	0	XX	0	1	0	0	1	1	
0001	X	00	00	10	0	0	XX	0	0	X	0	0	0	
0011	X	00	01	10	0	0	XX	0	0	X	0	0	0	
0010	X	10	01	00	0	0	XX	0	0	X	0	0	0	
1000	1	01	01	00	0	0	XX	1	0	X	0	0	0	
0101	1	XX	XX	XX	0	1	01	0	1	X	0	0	0	
0110	X	XX	XX	XX	0	0	XX	0	0	1	0	1	0	
0111	X	XX	XX	XX	0	0	XX	0	0	1	1	0	0	
1001	X	XX	XX	XX	0	1	00	0	0	X	0	0	0	
1011	X	XX	XX	XX	0	1	10	0	0	X	0	0	0	
1101	0	00	10	01	0	1	11	0	0	X	0	0	0	

Tabela 1: Tabela verdade das saídas do controle

Com base nesta tabela, utilizamos Mapas de Karnaugh para minimizar, na forma de soma de produtos, as expressões para as saídas. Os resultados obtidos estão exibidos a seguir, em que Q_i representa o i -ésimo bit do estado atual.

$$PCsrc = \overline{Q_3} Q_2 + Q_3 \overline{Q_2} \overline{Q_1}$$

$$ALUOp_1 = \overline{Q_3} Q_1 \overline{Q_0}, ALUOp_0 = Q_3 \overline{Q_2} \overline{Q_1}$$

$$ALUSrcA_1 = Q_2 \overline{Q_1} Q_0 + \overline{Q_3} \overline{Q_2} \overline{Q_1} \overline{Q_0}, ALUSrcA_0 = \overline{Q_3} Q_1 + Q_3 \overline{Q_2} \overline{Q_1}$$

$$ALUSrcB_1 = \overline{Q_3} Q_0, ALUSrcB_0 = Q_2 \overline{Q_1} Q_0 + \overline{Q_3} \overline{Q_2} \overline{Q_1} \overline{Q_0}$$

$$PCbackWrite = \overline{Q_3} \overline{Q_2} \overline{Q_1} \overline{Q_0}$$

$$RegWrite = Q_3 \overline{Q_2} Q_0 + Q_2 \overline{Q_1} Q_0$$

$$Mem2Reg_1 = \overline{Q_2} Q_1 Q_0 + Q_3 Q_2 \overline{Q_1} Q_0, Mem2Reg_0 = Q_2 \overline{Q_1} Q_0$$

$$PCWriteCond = Q_3 \overline{Q_2} \overline{Q_1} \overline{Q_0}$$

$$PCWrite = \overline{Q_3} \overline{Q_2} \overline{Q_1} \overline{Q_0} + \overline{Q_3} Q_2 \overline{Q_1} Q_0$$

$$IorD = \overline{Q_3} Q_1$$

$$MemWrite = \overline{Q_3} Q_2 Q_1 Q_0$$

$$MemRead = \overline{Q_3} Q_2 \overline{Q_1} \overline{Q_0} + \overline{Q_3} Q_2 Q_1 \overline{Q_0}$$

$$IRWrite = \overline{Q_3} \overline{Q_2} \overline{Q_1} \overline{Q_0}$$

Tendo essas equações, é fácil implementar a lógica de saída. O esquemático a seguir mostra essa implementação.

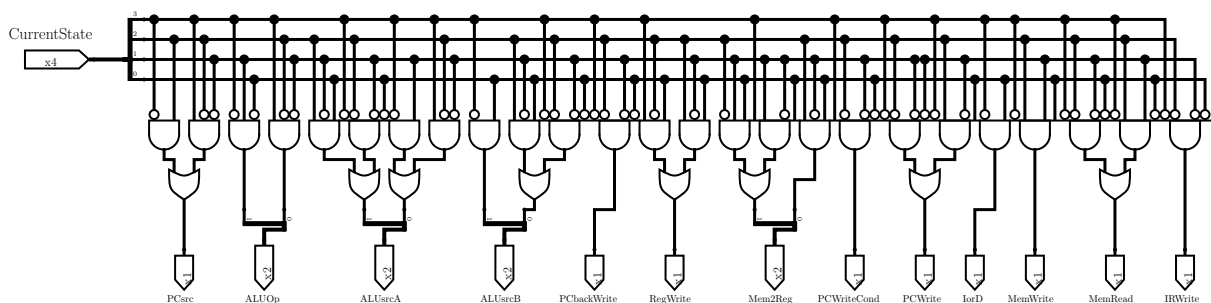


Figura 3: Esquemático da lógica de saída do controle

Estado Atual	Próximo Estado							
	0000011	0100011	0010011	1100111	0110011	1100011	1101111	0110111
0000	0001	0001	0001	0001	0001	0001	0001	0001
0001	0011	0011	0011	0011	0010	1000	0101	1101
0011	0110	0111	1001	0101	X	X	X	X
0010	X	X	X	X	1001	X	X	X
1000	X	X	X	X	X	0000	X	X
0101	X	X	X	0000	X	X	0000	X
0110	1011	X	X	X	X	X	X	X
0111	X	0000	X	X	X	X	X	X
1001	X	X	0000	X	0000	X	X	X
1011	X	0000	X	X	X	X	X	X
1101	X	X	X	X	X	X	X	0000

Tabela 2: Tabela de transição de estados do controle

A abordagem para implementar a lógica de próximo estado é um pouco diferente; não minimizaremos equações nem nada do tipo. Na verdade, tendo definido que um registrador de deslocamento será utilizado como elemento de memória, montamos outra tabela a partir da Tabela 2; uma em que, ao invés do próximo estado, representamos, em cada célula, a operação que o registrador precisa fazer para chegar àquele estado.

O Registrador de Deslocamento é capaz de fazer quatro operações: *shift left(L)*, *shift left(R)*, *load(D)* e *reset*, em que a variável entre parenteses representa o argumento necessário para aquela operação (L: o bit que entra a esquerda no *shift left*, R: o bit que entra a direita *shift left*, D: o vetor de quatro bits que substitui o estado atual no *load*). O *reset* não foi utilizado nesse projeto, ao invés dele, é utilizado o *load(0000)*.

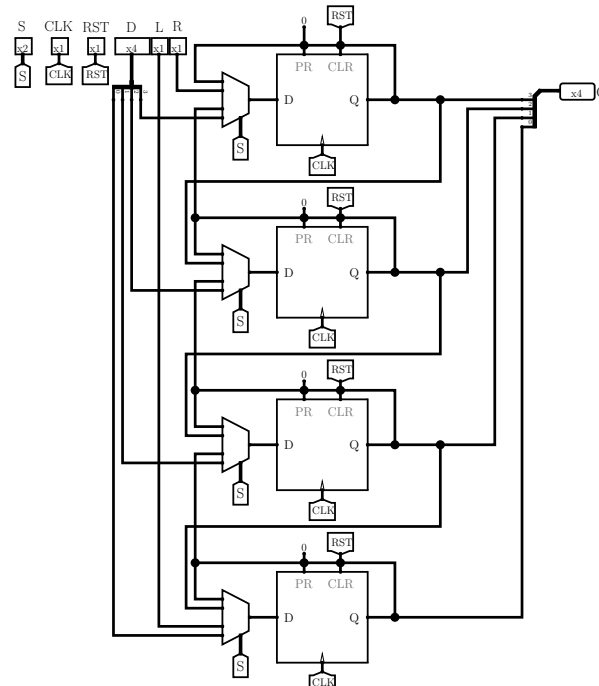


Figura 4: Lógica interna do Registrador de Deslocamento

Estado Atual	Próximo Estado							
	0000011	0100011	0010011	1100111	0110011	1100011	1101111	0110111
0000	SL(1)	SL(1)	SL(1)	SL(1)	SL(1)	SL(1)	SL(1)	SL(1)
0001	SL(1)	SL(1)	SL(1)	SL(1)	SL(0)	SR(1)	LOAD(0101)	LOAD(1101)
0011	SL(0)	SL(1)	SR(1)	LOAD(0101)	X	X	X	X
0010	X	X	X	X	SR(1)	X	X	X
1000	X	X	X	X	X	SL(0)	X	X
0101	X	X	X	LOAD(0000)	X	X	LOAD(0000)	X
0110	SR(1)	X	X	X	X	X	X	X
0111	X	LOAD(0000)	X	X	X	X	X	X
1001	X	X	LOAD(0000)	X	LOAD(0000)	X	X	X
1011	X	LOAD(0000)	X	X	X	X	X	X
1101	X	X	X	X	X	X	X	LOAD(0000)

Tabela 3: Tabela de transição de estados do controle

Note que os estados foram a priori escolhidos de forma a minimizar os ‘LOAD’s necessários - claro que não há garantia de que essa minimização foi ótima, ela foi feita por inspeção.

Agora, basta utilizar um multiplexador de tamanho adequado para selecionar as entradas corretas para o registrador de deslocamento para cada caso. Suas

2.2 Tarefa IV

Enunciado

2.2.1 Tarefa V

Enunciado

2.2.2 Tarefa VI

Enunciado

2.3 Tarefa VII

Enunciado

Enunciado

Enunciado

Enunciado

Enunciado